Arcade Night: Streamlit lab

You are planning a visit to a fictional arcade. Build a page that finds games in your chosen category that cost no more than your token limit. Show the matching games, their count, mean rating and a ratings chart. Work individually in app_starter.py.

Start the app

Extract the entire ZIP. Open a terminal in the extracted folder and use your lecture Python environment.

```
python -m pip install -r requirements.txt
python setup_check.py
python -m streamlit run app_starter.py
```

Open the Local URL. Keep the terminal running. Save each change and rerun when prompted. The starter already displays a table and placeholders. Stop with Ctrl+C.

How the page updates

When you change a widget, Streamlit reruns the script from the top. The widget returns its current value to category or budget. The filter uses those values. The count, mean, table and chart then display the new result. You do not need a Submit button.

Part 1: Introduce your arcade

At TODO 1, add a short caption. You may choose your own wording. The title, CSV loading and data checks are supplied.

```
st.caption("Find a game for your budget. All games and ratings are fictional.")
```

The CSV has 12 fictional games: Game, Category, Tokens per play, and Rating out of 5. The budget applies to **one game**.

Part 2: Choose a category and budget

Replace the fixed assignments at TODOs 2a-2b. The category condition is supplied. At TODO 2c, apply the token condition to the rows it leaves behind.

```
category = st.selectbox("Game category", CATEGORIES)
budget = st.slider("Maximum tokens per game", 0, 10, 4)
# At TODO 2c, after the supplied category condition:
filtered = filtered[filtered["Tokens"] <= budget]
```

The <= comparison includes games exactly at the budget. Before running Action / 4, write your predicted game names in test_checklist.txt. Costs: Laser Dash 3, Zombie Pop 4, Dragon Quest 7, Space Defender 6. Then run it and compare.

Part 3: Show the count and mean rating

At TODO 3a, add the count. Inside the supplied else block, replace its placeholder with the two mean-rating lines. Keep the indentation.

```
st.metric("Games available", len(filtered))
# Inside else:
mean_rating = filtered["Rating"].mean()
st.metric("Mean player rating", f"{mean_rating:.2f} / 5")
```

For Action / 4, the ratings are 4.6 and 4.2. Their mean is (4.6 + 4.2) / 2 = 4.40. The .2f format displays two decimal places. Keep the empty-result warning.

Part 4: Compare the games

The table and chart-data preparation are supplied. Replace the chart placeholder at TODO 4 with this line inside the existing if block:

```
st.bar_chart(chart_data,
             y_label="Fictional player rating (out of 5)")
```

One bar represents one matching game. The count, mean, table and chart must all use the same filtered rows. Compare your app with examples/02_action_4.png and Example_Screenshots.pdf. Chart order and ordinary colors may differ.

Try the dashboard

Complete test_checklist.txt. All / 0 should show zero games, a warning and an empty table, with no mean or chart. All / 10 restores the full list.

One small change on your own

Change the slider starting value from 4 to 6 without copying another code example. Predict the All-category result in test_checklist.txt first. Save, stop the server with Ctrl+C, close its browser tab, restart and open the Local URL in a new tab. Check the slider starts at 6 and compare the result with your prediction. Keep this change in your submitted script. Then select Action / 4 for the required screenshot.

Answer the three questions in reflections.txt using one or two sentences each. You may discuss your game choice with a classmate.

Canvas submission

Submit **COSC221_Arcade_YourName.zip** by the Canvas deadline. Include:

1. Your completed app_starter.py.
2. requirements.txt.
3. data/arcade_games.csv, unchanged.
4. Completed test_checklist.txt and reflections.txt.
5. dashboard_action_4.png: your own running app showing the category, budget, both metrics and matching table for Action / 4.

Use your computer screenshot tool. The chart may sit below the captured area. Leave out virtual environments and installed packages. Submit one dashboard with the starting-budget change. No additional feature is required.

Marking guide (100 points)

Page introduction and correct data: 10.
Category selector, budget slider and correct filtering: 30.
Count and mean from matching games: 20.
Table, chart and empty-result behavior: 20.
Predictions, starting-budget change, checks, screenshot and explanations: 20.

If something goes wrong

No page: launch with python -m streamlit run.
CSV missing: extract the whole folder, keeping data/ beside the script.
Old results: save and rerun.
Games cost too much: use <= for the budget condition.
Category filter disappears: apply the budget condition to filtered.
Indentation error: mean lines belong inside else, chart inside if.
Empty at 0 tokens: expected. Increase the budget.